

搜索策略

祝恩
计算机学院
国防科技大学



1. 盲目搜索策略：
宽度优先, 一致代价, 深度优先,
深度受限, 迭代加深, 双向搜索
2. 启发式搜索策略：
贪婪最佳优先搜索, A^* 搜索

- ① BFS: Breadth-first search
- ② UCS: Uniform-cost search
- ③ DFS: Depth-first search
- ④ DLS: Depth-limited search
- ⑤ IDS: Iterative deepening search
- ⑥ BS: Bidirectional search

1 盲目搜索策略





(1)宽度优先

宽度优先搜索BFS(树搜索)

```
function Breadth-First-Search(problem) returns a solution, or failure
  node  $\leftarrow$  a node with State=problem.Initial-State, Path-Cost=0
  if problem.Goal-Test(node.State) then return Solution(node)
  frontier  $\leftarrow$  {node}

  loop do
    if Empty?(frontier) then return failure
    node  $\leftarrow$  POP the shallowest node in frontier
    add node.State to explored
    for each action in problem.Actions(node.State) do
      child  $\leftarrow$  Child-Node(problem, node, action)

      if problem.Goal-Test(child.State) then return Solution(child)
      frontier  $\leftarrow$  Insert(child, frontier)
```

提前进行测试：在加入frontier之前进行测试，为什么？

宽度优先搜索BFS(图搜索)

```
function Breadth-First-Search(problem) returns a solution, or failure
  node  $\leftarrow$  a node with State=problem.Initial-State, Path-Cost=0
  if problem.Goal-Test(node.State) then return Solution(node)
  frontier  $\leftarrow$  {node}
  explored  $\leftarrow$  an empty set
  loop do
    if Empty?(frontier) then return failure
    node  $\leftarrow$  POP the shallowest node in frontier
    add node.State to explored
    for each action in problem.Actions(node.State) do
      child  $\leftarrow$  Child-Node(problem, node, action)
      if child.State is not in explored or frontier then
        if problem.Goal-Test(child.State) then return Solution(child)
        frontier  $\leftarrow$  Insert(child, frontier)
      [else delete child]
```



(2)一致代价搜索UCS

UCS算法(图搜索)

```

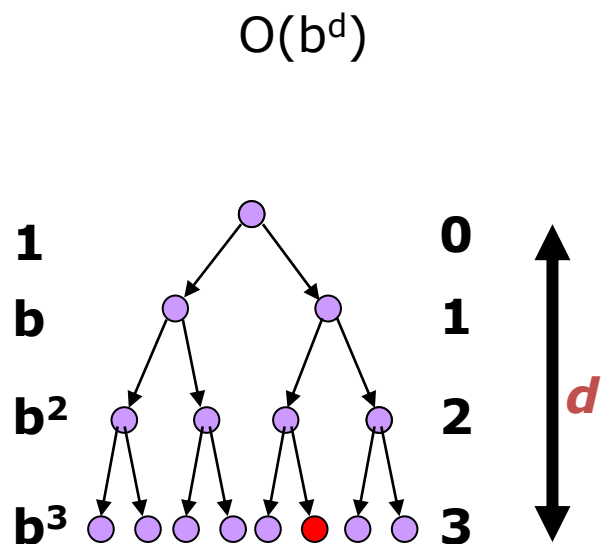
function Uniform-Cost-Search(problem) returns a solution, or failure
  node  $\leftarrow$  a node with State=problem.Initial-State, Path-Cost=0
  if problem.Goal-Test(node.State) then return Solution(node)
  frontier  $\leftarrow$  {node}
  explored  $\leftarrow$  an empty set
  loop do
    if Empty?(frontier) then return failure
    node  $\leftarrow$  POP the lowest-cost node in frontier
    if problem.Goal-Test(child.State) then return Solution(child)
    add node.State to explored
    for each action in problem.Actions(node.State) do
      child  $\leftarrow$  Child-Node(problem, node, action)
      if child.State is not in explored or frontier then
        frontier  $\leftarrow$  Insert(child, frontier)
      else if child.State is in frontier with higher Path-Cost then
        replace that frontier node with child
      [else delete child]
  
```

If child.State is explored, ...

正常测试(不能提前): 从**frontier**中移出后,先目标测试, 然后生成子结点.**why?**

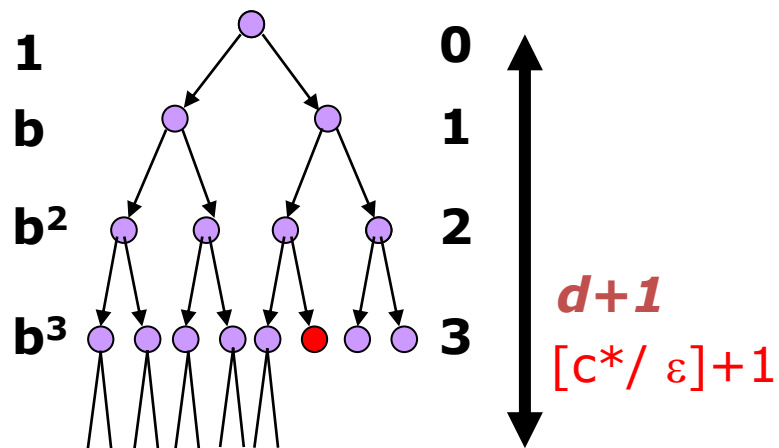
UCS Graph Search: 新结点的路径代价一定高于任何**explored**结点

对比BFS与UCS的复杂性



BFS

$O(b^{\lceil c^*/\epsilon \rceil + 1})$



UCS

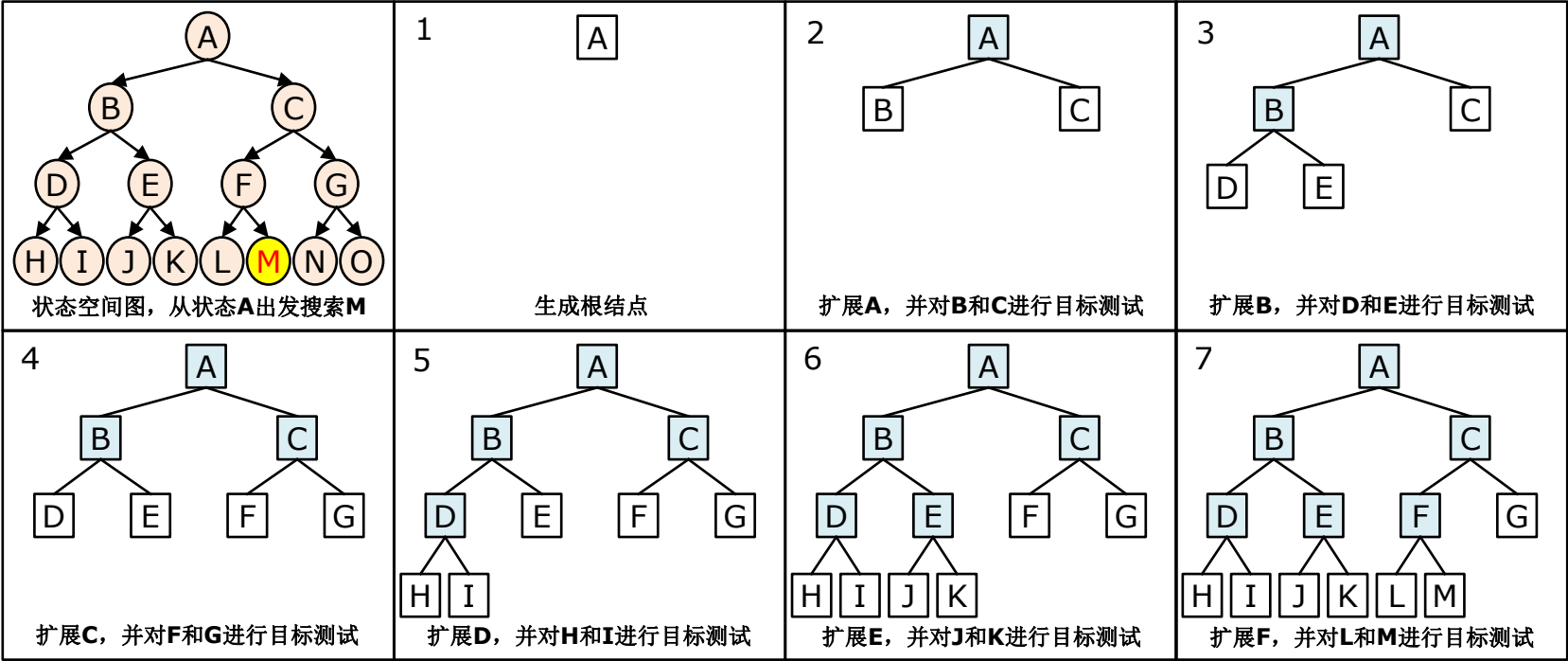
假设每步代价相同

宽度优先搜索和一致代价搜索
都有很大的空间复杂度



(3)深度优先搜索DFS

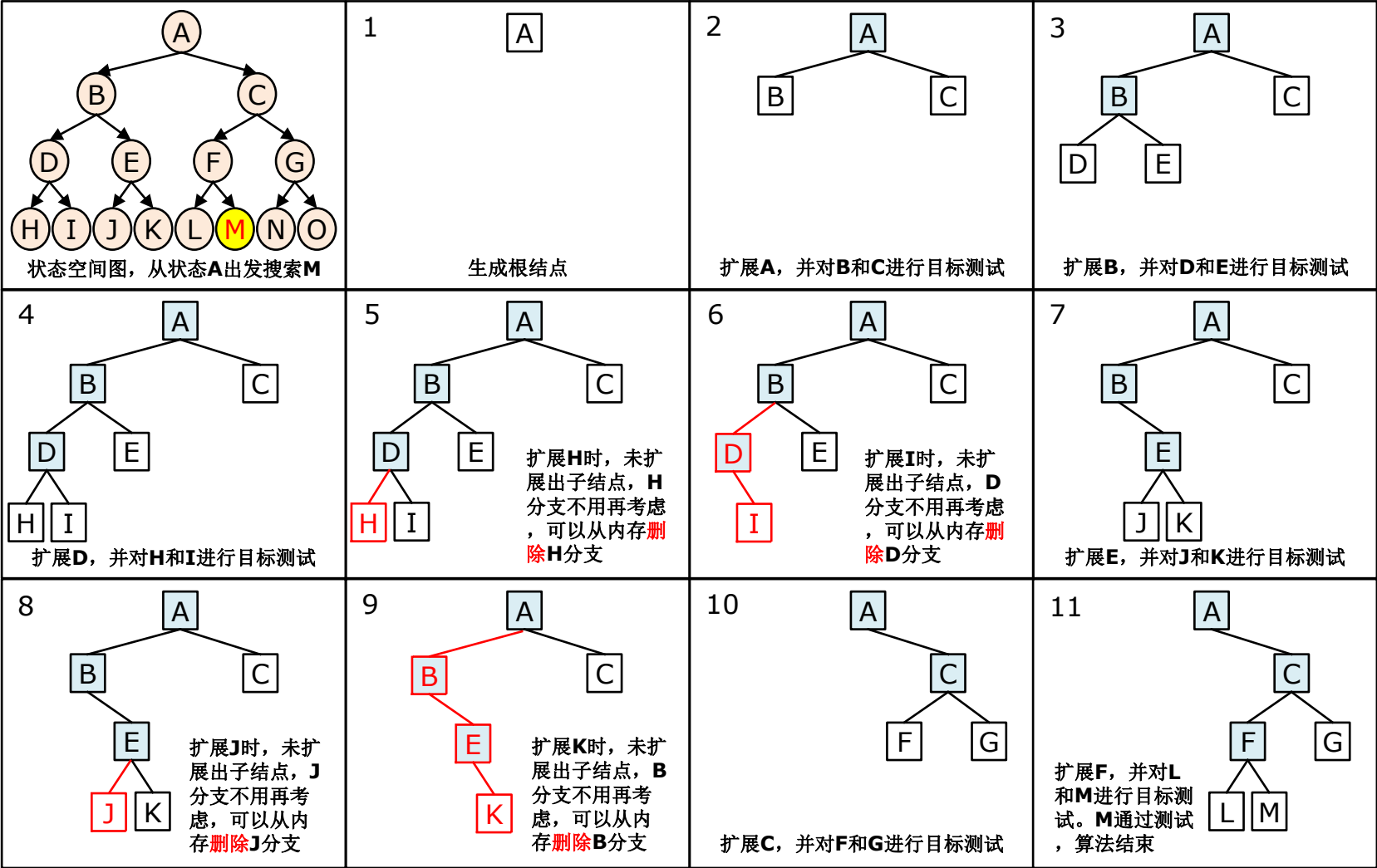
BFS vs. DFS



宽度优先搜索

space complexity is $O(b^d)$, b is the branching factor, d is the goal depth.

BFS vs. DFS



深度优先搜索

space complexity is $O(bm)$, b is the branching factor, m is the largest depth.

目标测试

- 正常测试（扩展时测试）
 - 提前测试（生成时测试）
-
- 和宽度优先一样可进行提前测试

DFS的完备性、最优性、复杂性

☐ Complete?

	finite space	infinite space
depth first tree search	no	no
depth first graph search	complete	no

☐ not Optimal

☐ b: branching factor

☐ d: depth of shallowest goal node

☐ m: maximal depth of a leaf node

☐ Number of nodes generated (worst case):
 $1 + b + b^2 + \dots + b^m = O(b^m)$

☐ Time complexity is $O(b^m)$

☐ Space complexity is $O(bm)$

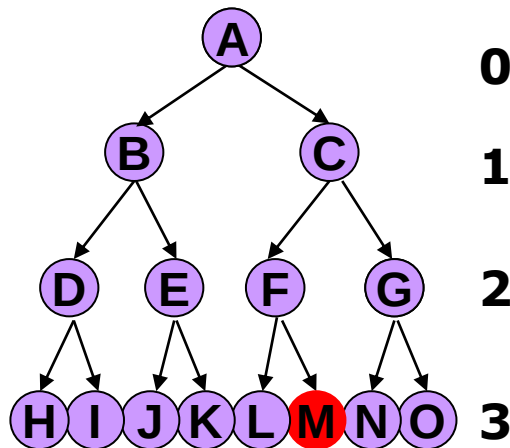


(4)深度受限搜索DLS

深度受限搜索DLS

Depth-Limited Search

- Depth-first with **cutoff** k depth below which nodes are not expanded
- **Recursive-DLS**($node, problem, limit$) returns:
 - Solution
 - Failure (no solution) (没有达到限定深度, 也没有找到解) (加大深度限定也一定无解)
 - Cutoff (no solution within cutoff) (虽然达到了限定深度, 但仍没有找到解) (加大深度限定可能有解)



Recursive-DLS(A,problem,2) returns _____

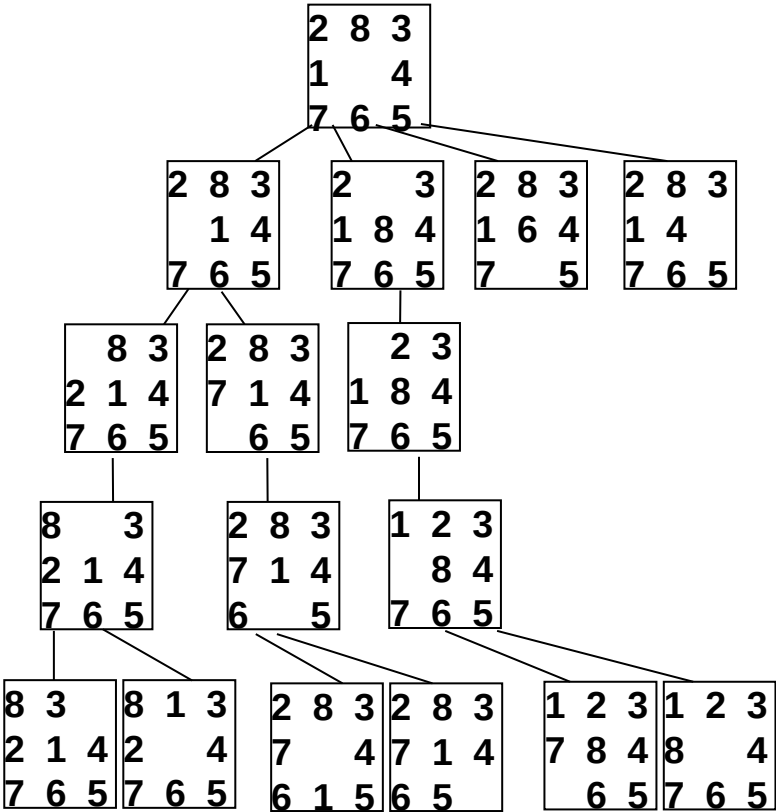
Recursive-DLS(A,problem,3) returns _____

Recursive-DLS(B,problem,2) returns _____

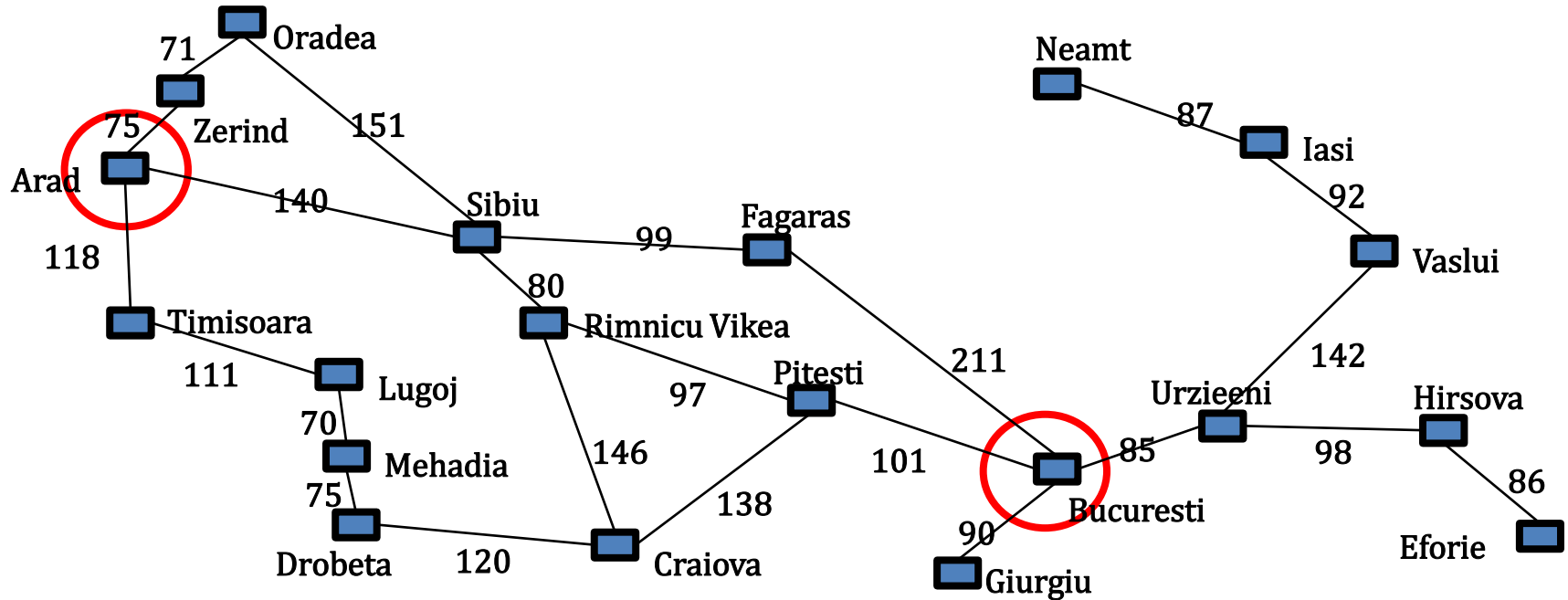
Recursive-DLS(B,problem,3) returns _____

深度受限搜索实例

□ 限定深度4



实例：确定受限的深度值



- Only 20 cities on the map
- Any city can be reached from any other city in less than 10 steps

DLS的完备性、最优性、复杂性

- ❑ not complete
- ❑ not optimal
- ❑ L: depth limit
- ❑ time complexity $O(b^L)$,
the number of nodes generated.
- ❑ space complexity $O(bL)$



(5)迭代加深搜索IDS

迭代加深搜索IDS

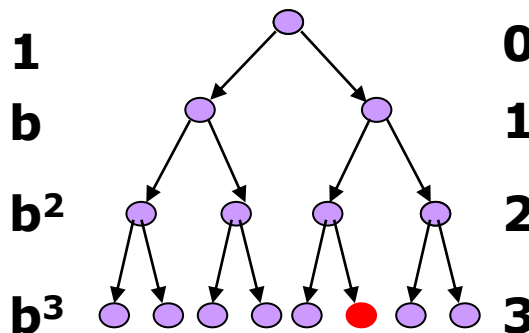
Iterative Deepening Search

- Iterative Deepening Search(IDS)
 - Provides the best of both breadth-first and depth-first search

```
function Iterative-Deepening-Search(problem) returns a solution, or failure
for depth = 0 to  $\infty$  do
    result  $\leftarrow$  Depth-Limited-Search(problem, depth)
    if result  $\neq$  cutoff then return result
```

IDS的完备性、最优性、复杂性

- ❑ **complete** if the branching factor is finite.
- ❑ **optimal** if the path cost is a nondecreasing function of the depth of the node
- ❑ d : the depth of the shallowest goal node.
- ❑ space complexity: $O(bd)$
- ❑ **how many nodes are going to be generated?**



IDS生成多少结点?

□ suppose $d=3$

□ Depth Limited Search

$$1+b+b^2+b^3$$

in general: $N_{DLS}=1+b+b^2+\dots+b^d=O(b^d)$

□ Iterative Deepening Search

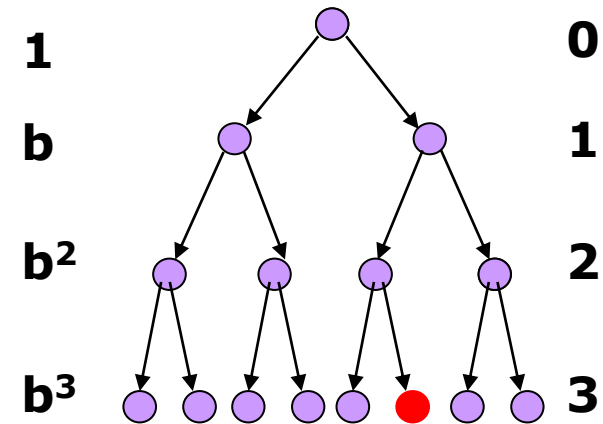
$$1 \quad \text{limit}=0$$

$$1+b \quad \text{limit}=1$$

$$1+b+b^2 \quad \text{limit}=2$$

$$1+b+b^2+b^3 \quad \text{limit}=3$$

total in general: $N_{IDS}=(d+1)1+db+(d-1)b^2+\dots+1b^d = O(b^d)$



IDS生成多少结点?

- For $b = 10, d = 5$,
 - $N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$
 - $N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$
- Overhead
 - $(123,456 - 111,111) / 111,111 = 11\%$

In general, iterative deepening is the preferred uninformed search method when there is a large search space and the depth of the solution is not known.

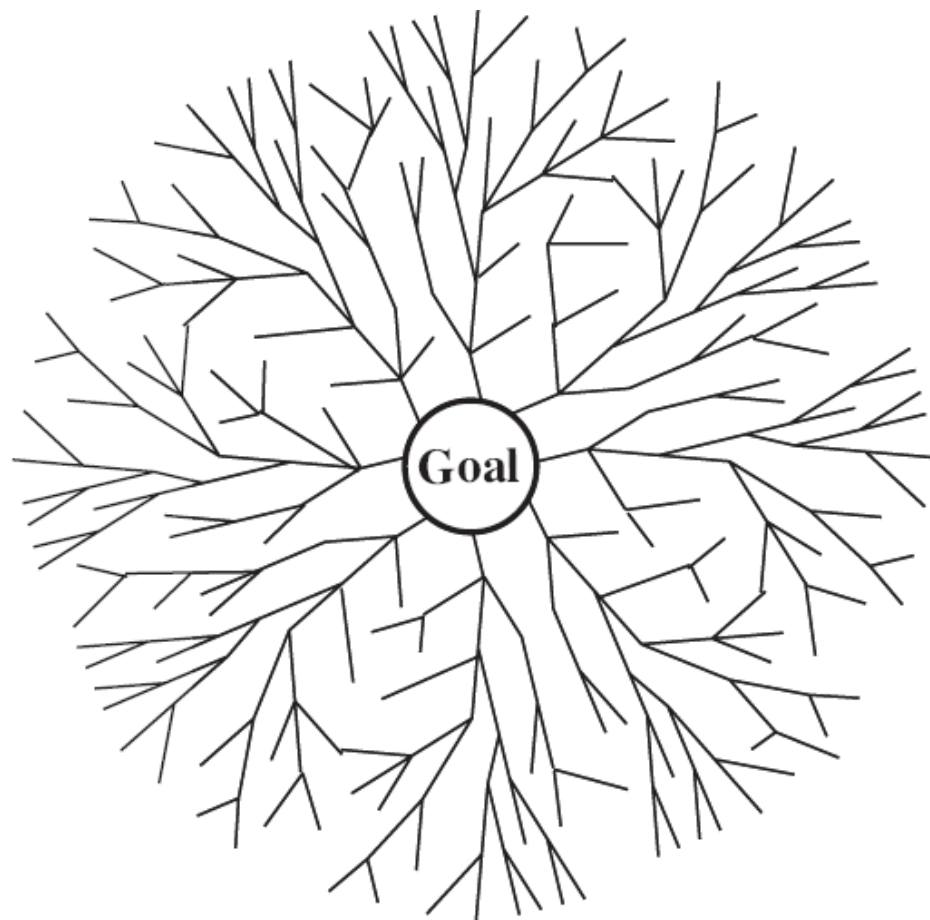
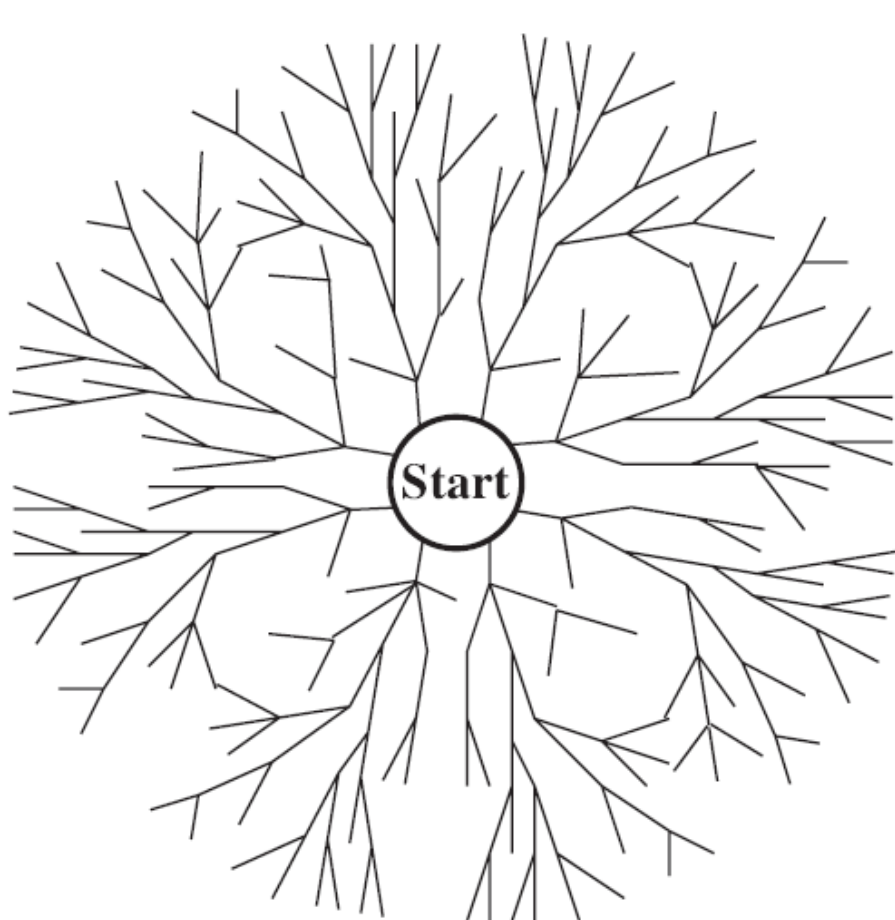


(6)双向搜索

双向搜索

Bi-directional search

□ $b^{d/2} + b^{d/2}$ is much less than b^d



启发式搜索与盲目搜索的本质区别是什么？

贪婪最佳优先搜索

A*搜索

2 启发式搜索策略



想法：估计到目标的距离

- It would be better if the agent knew whether or not the city it is travelling to gets it closer to Bucuresti
- Of course, the agent **doesn't know the exact distance** or path to Bucuresti (it wouldn't need to search otherwise!)
- The agent must **estimate the distance** to Bucuresti

这种估计称为启发式函数

- More generally
 - We want the search algorithm to be able to **estimate** the path cost from the current node to the goal
- This estimate is called a **heuristic function** (启发式函数)
- Can't be done based on **problem formulation**
 - Need to add **additional** information
 - Informed search

1. S_0
2. $ACTIONS(s)$
3. $RESULT(s,a)$
4. $Goal-Test(s)$
5. $Action-Cost$

启发式函数 $h(n)$

□ Heuristic function $h(n)$

- $h(n)$: estimated cost from node n to goal
- depends only on $STATE(n)$ and the goal state $GOAL$.
- $h(n_1) < h(n_2)$ means it's **probably** cheaper to get to the goal from n_1
- $h(n_{goal}) = 0$

1. S_0
2. $ACTIONS(s)$
3. $RESULT(s,a)$
4. $Goal-Test(s)$
5. $Action-Cost$
6. **Heuristic: $h(.)$**

评估函数 $f(n)$

- $h(n)$: Heuristic function
 - estimated cost from node n to goal
- $g(n)$: Path Cost
 - cost from the initial node to n
- $f(n)$: Evaluation function
 - $f(n) = g(n)$ UCS (Uniform cost)
 - $f(n) = h(n)$ Greedy Best-First
 - $f(n) = g(n) + h(n)$ A*

 - $f(n) = \text{depth}(n)$ BFS
 - $f(n) = -\text{depth}(n)$ DFS

启发式函数的实例

□ $h_1(N)$ = number of misplaced numbered tiles = 6

5		8
4	2	1
7	3	6

STATE(n)

1	2	3
4	5	6
7	8	

Goal state

启发式函数的实例

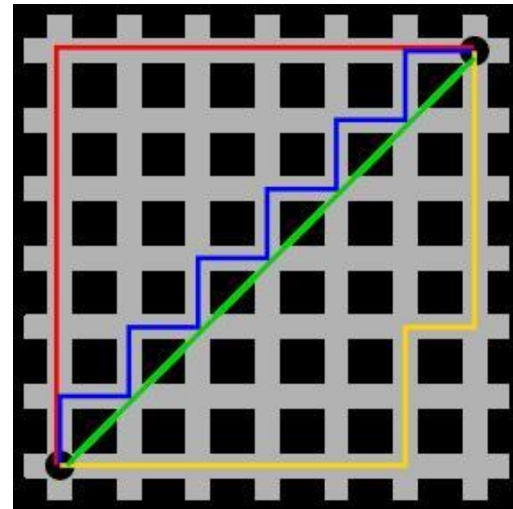
- $h_2(N)$ = sum of the (Manhattan) distance of every numbered tile to its goal position
 = $3+1+3+0+2+1+0+3=13$

5		8
4	2	1
7	3	6

STATE(n)

1	2	3
4	5	6
7	8	

Goal state



P1(x1, y1)到P2 (x2, y2)
 的曼哈顿距离:

$$|x1 - x2| + |y1 - y2|$$



贪婪最佳优先搜索

贪婪最佳优先搜索GBFS

Greedy best-first search

- Greedy best-first search expands the node that is closest to the goal. It evaluates nodes by using just the heuristic function; that is, $f(n)=h(n)$.

5		8
4	2	1
7	3	6

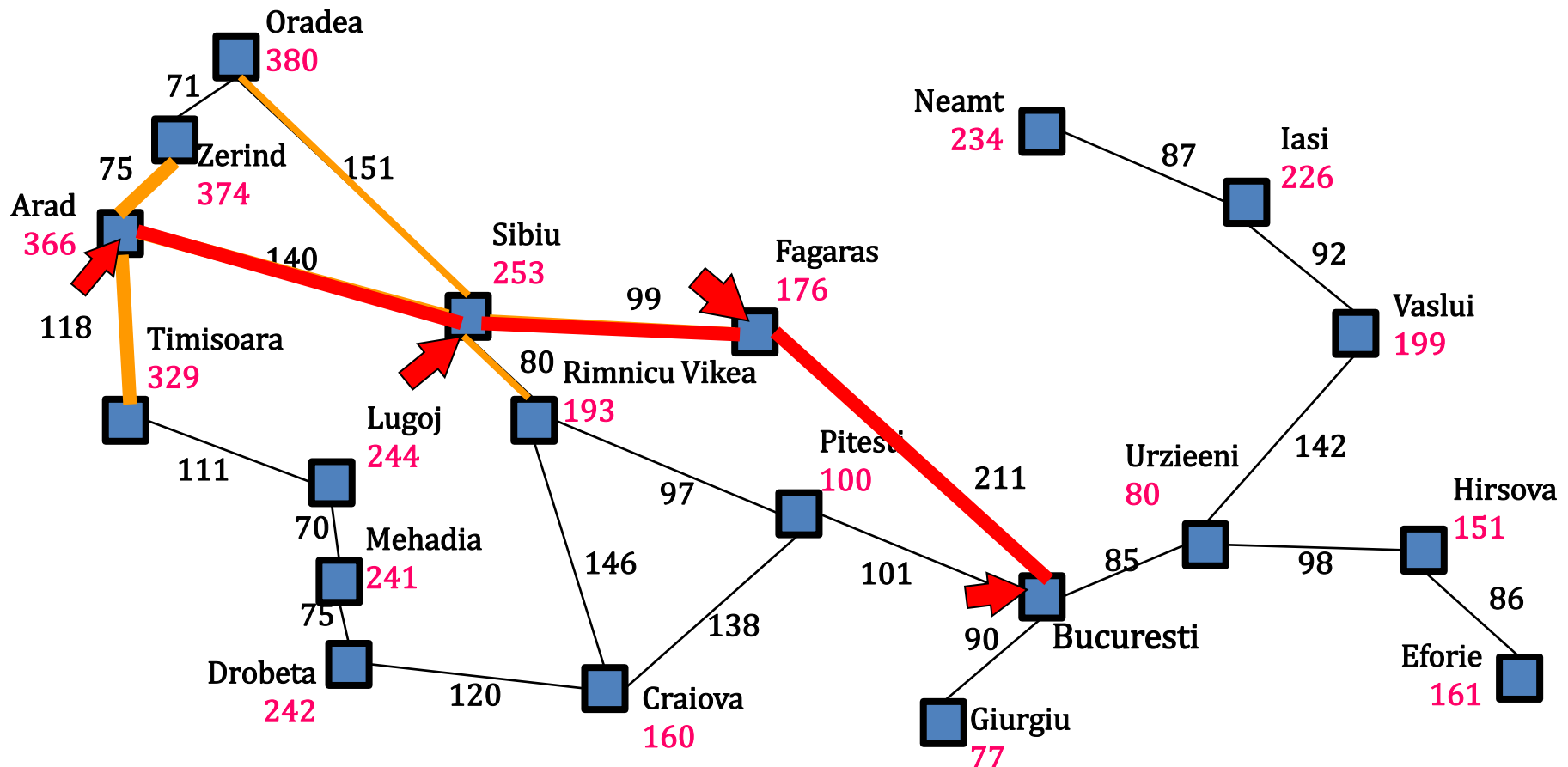
STATE(N)

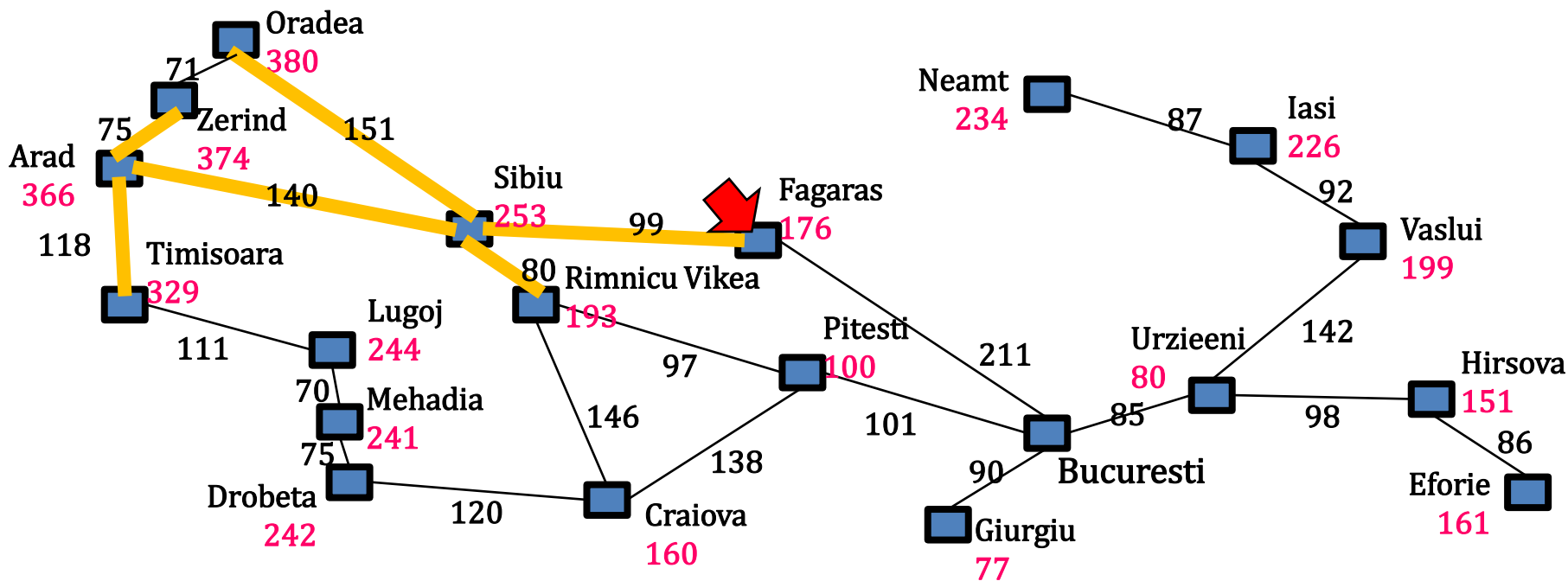
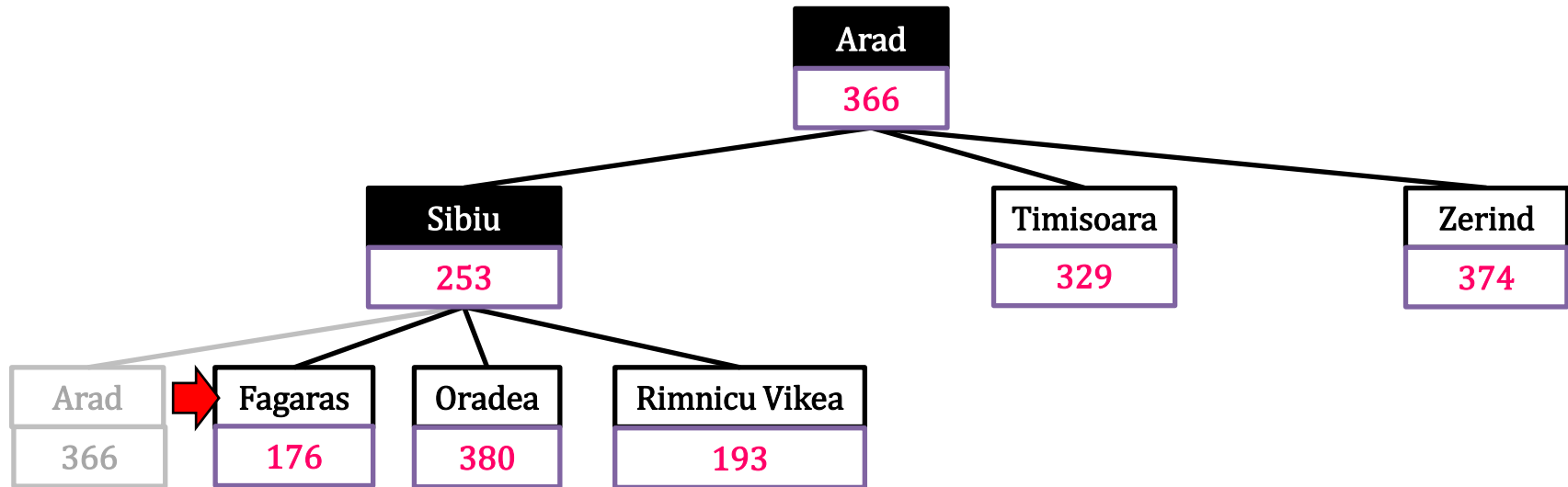
1	2	3
4	5	6
7	8	

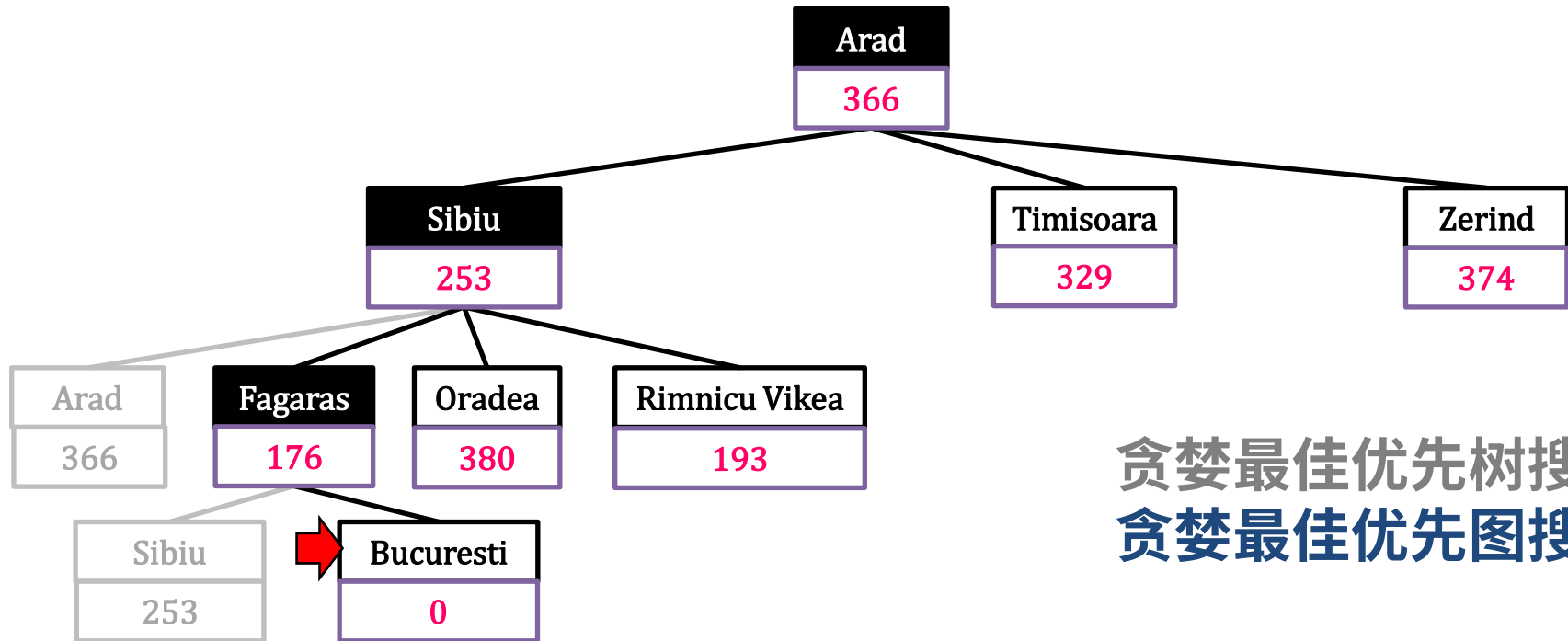
Goal state

GBFS实例：Romania问题

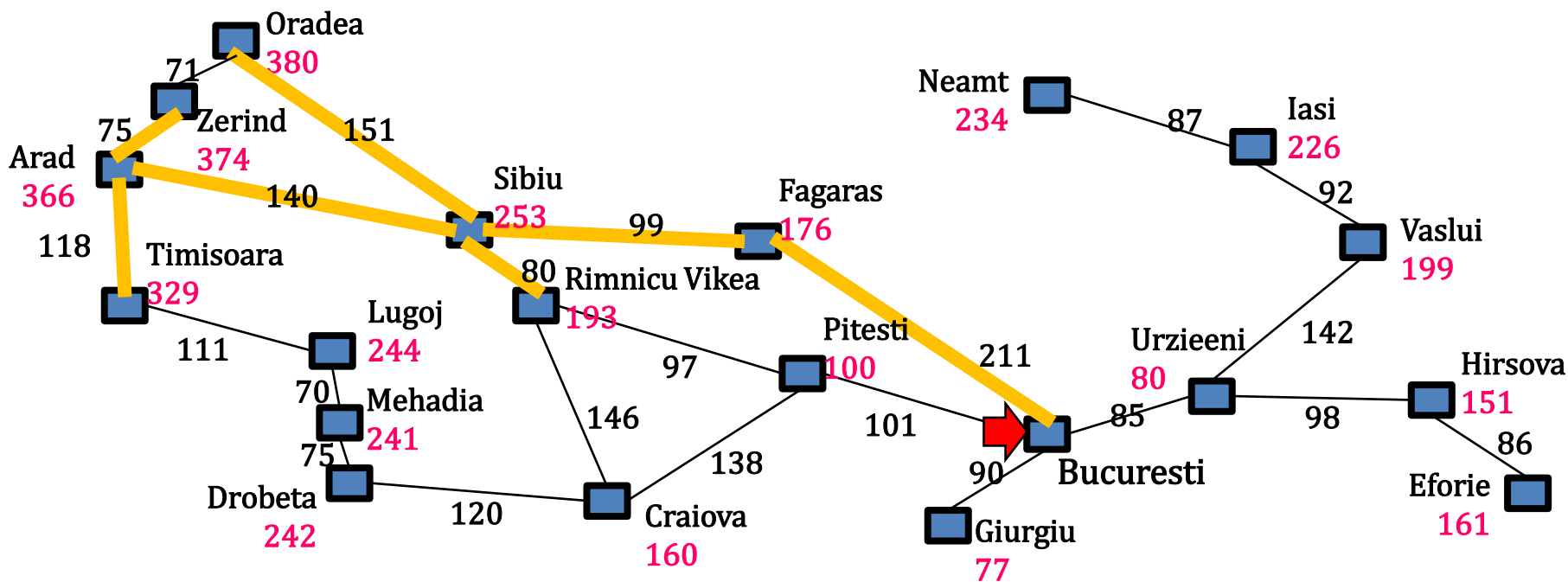
- Expand the node that seems closest to Bucuresti
- $h(n)$ = straight-line distance







贪婪最佳优先树搜索
贪婪最佳优先图搜索

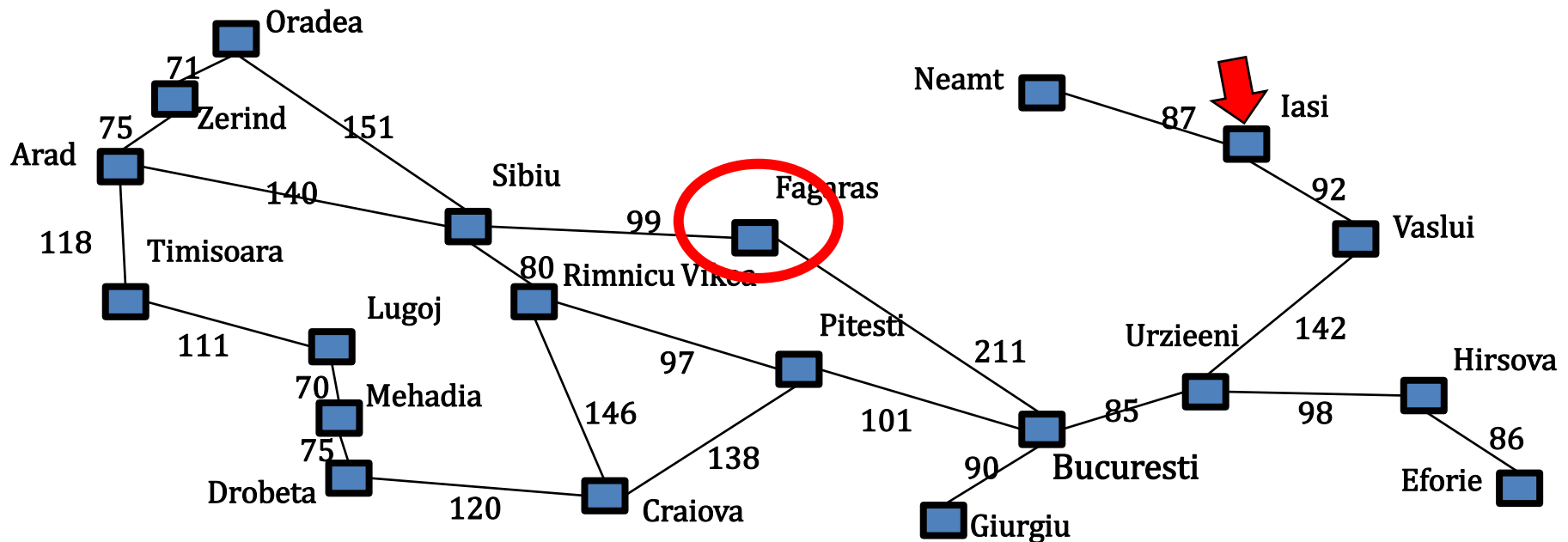
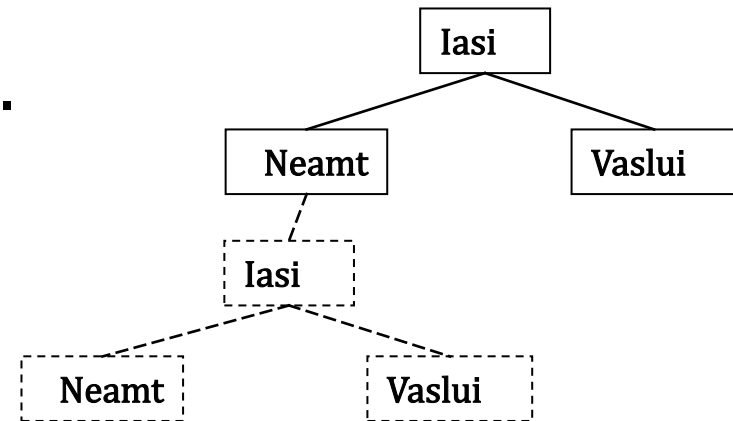




贪婪最佳优先树搜索
有何问题？

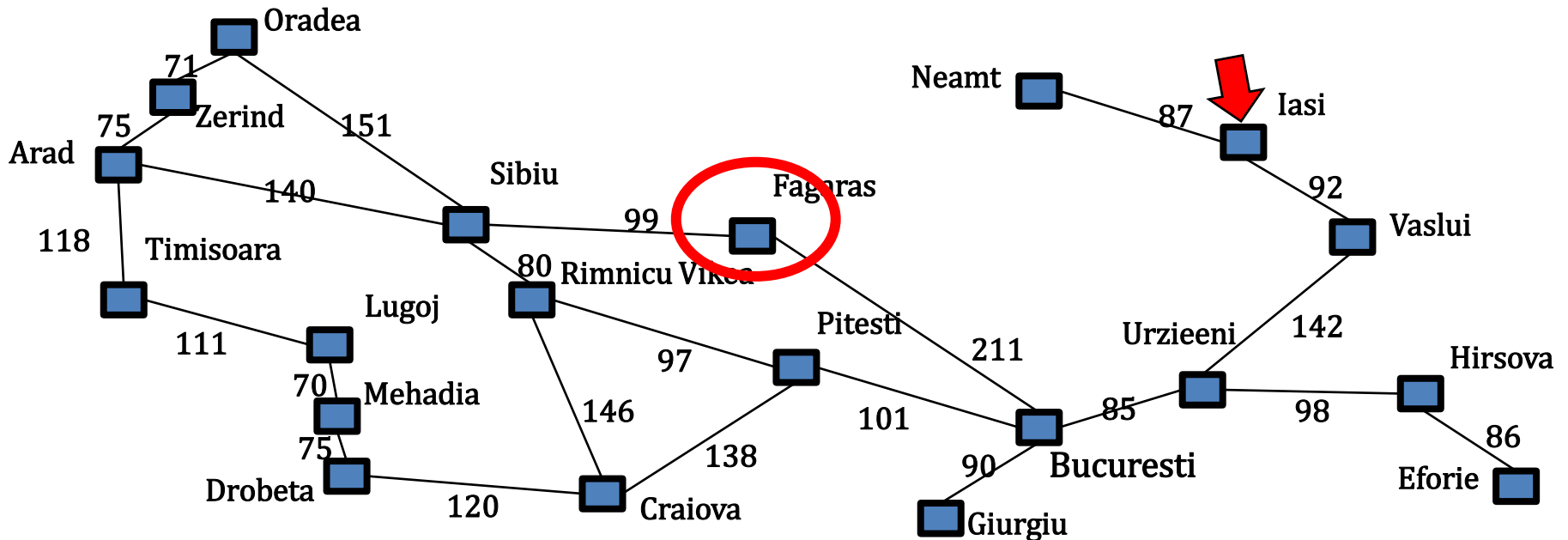
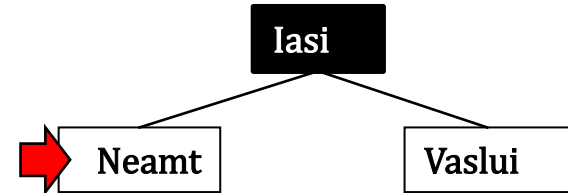
贪婪最佳优先树搜索

- try: from Iasi to Fagaras.
- can **stuck in loops**
(不是完备的)



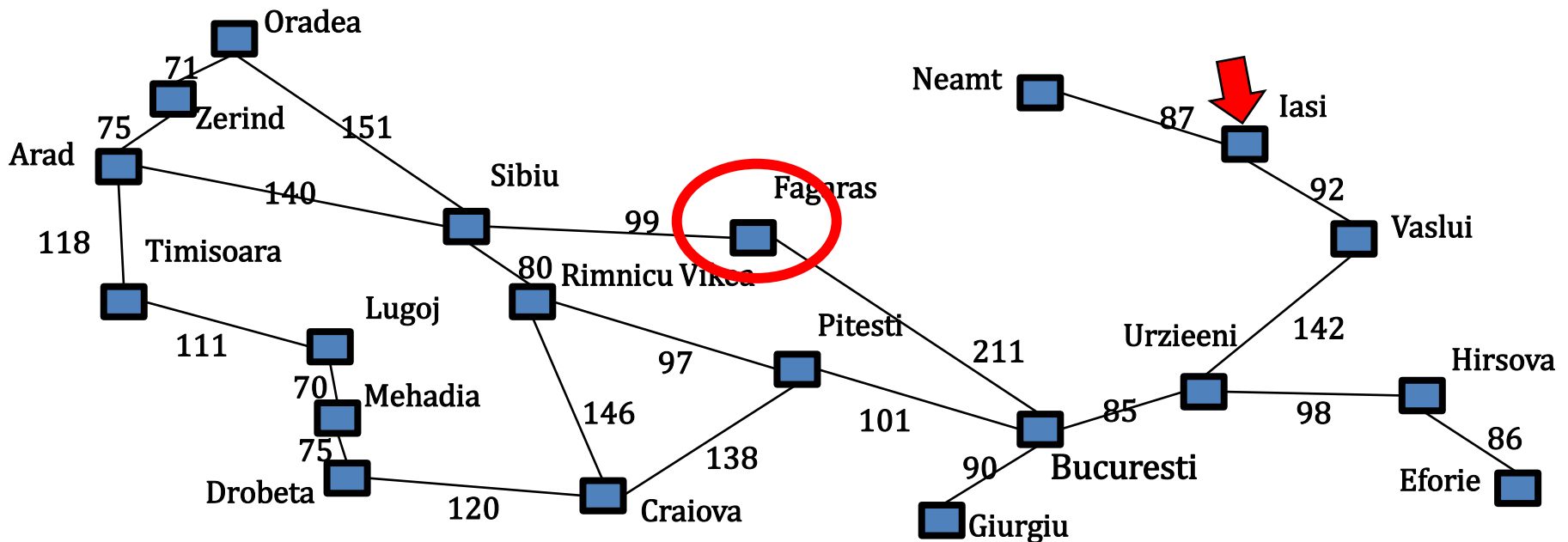
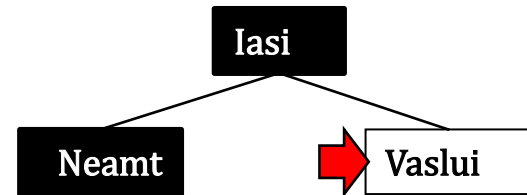
贪婪最佳优先图搜索

- try: from Iasi to Fagaras.
- consider checking repeated states



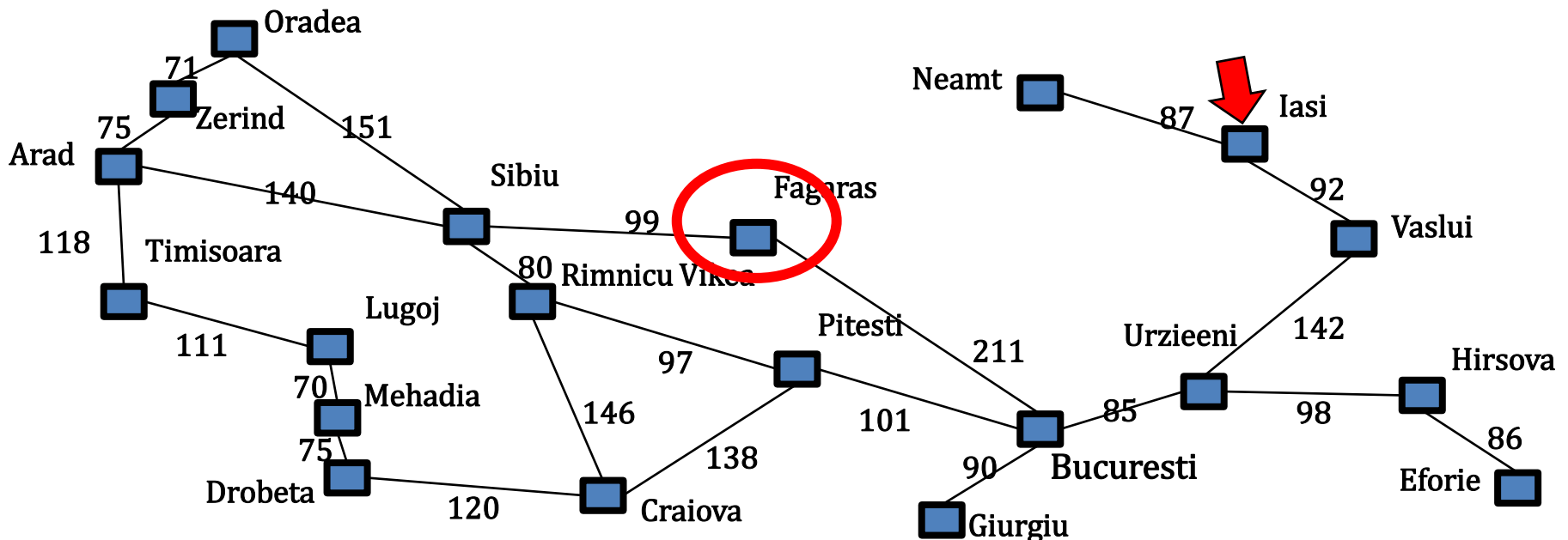
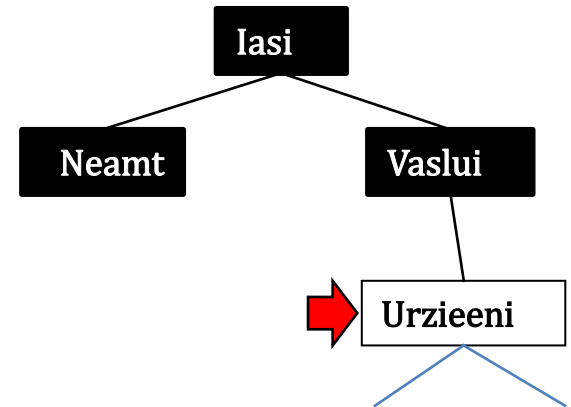
贪婪最佳优先图搜索

- try: from Iasi to Fagaras.
- consider checking repeated states



贪婪最佳优先图搜索

- try: from Iasi to Fagaras.
- consider checking repeated states



GBFS的完备性、最优性、复杂性

□ Complete?

	Finite space	Infinite space
GBFS tree search	incomplete (can get stuck in loops)	incomplete
GBFS graph search	complete	incomplete

GBFS的完备性、最优性、复杂性

☐ Optimal?

- No

☐ m: maximal depth of a leaf node

☐ Time

- $O(b^m)$, but a good heuristic can give dramatic improvement

☐ Space

- $O(b^m)$ -- keeps all nodes in memory

$$f(n) = g(n) + h(n)$$



A*搜索

A* 搜索

□ Idea

- avoid expanding paths that are already expensive

□ Evaluation function $f(n) = g(n) + h(n)$

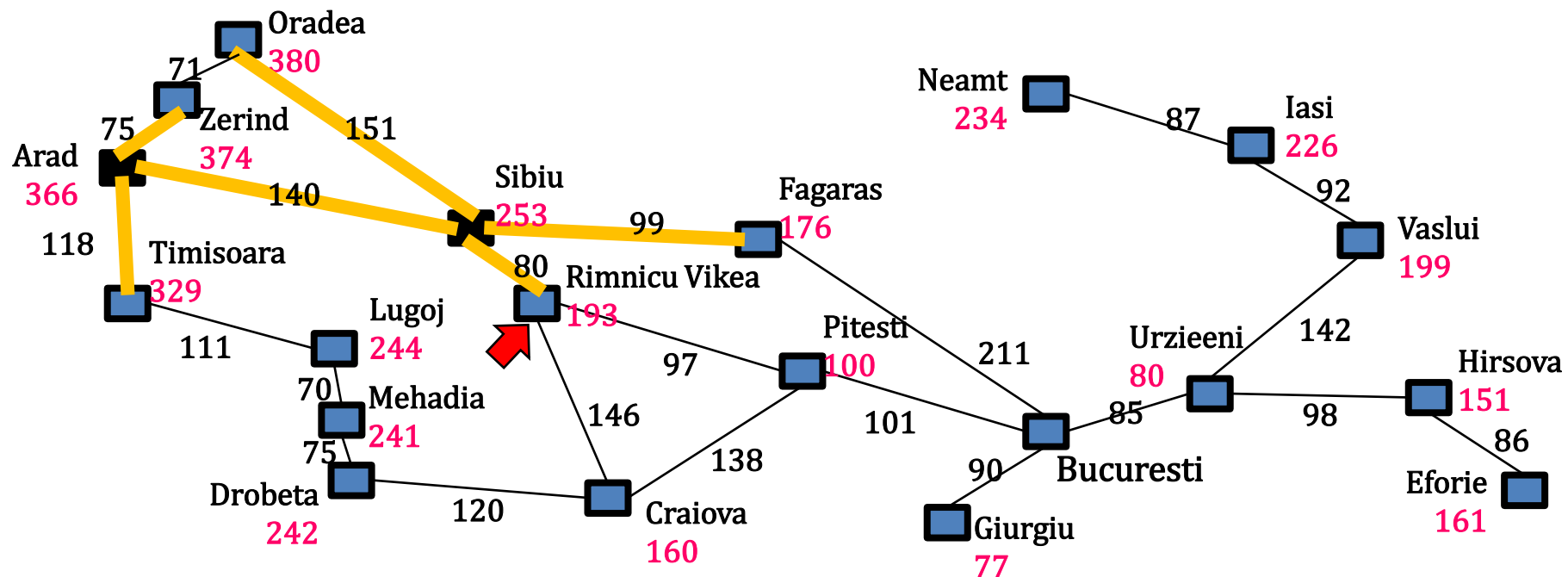
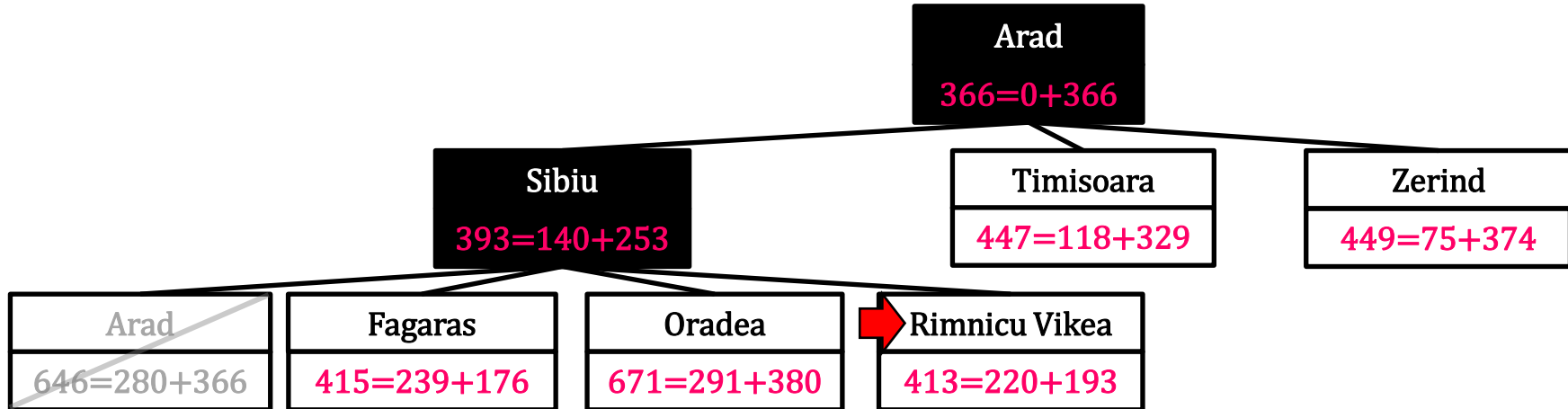
- $g(n)$ = cost so far to reach n
- $h(n)$ = estimated cost from n to goal
- $f(n)$ = estimated total cost of path through n to goal

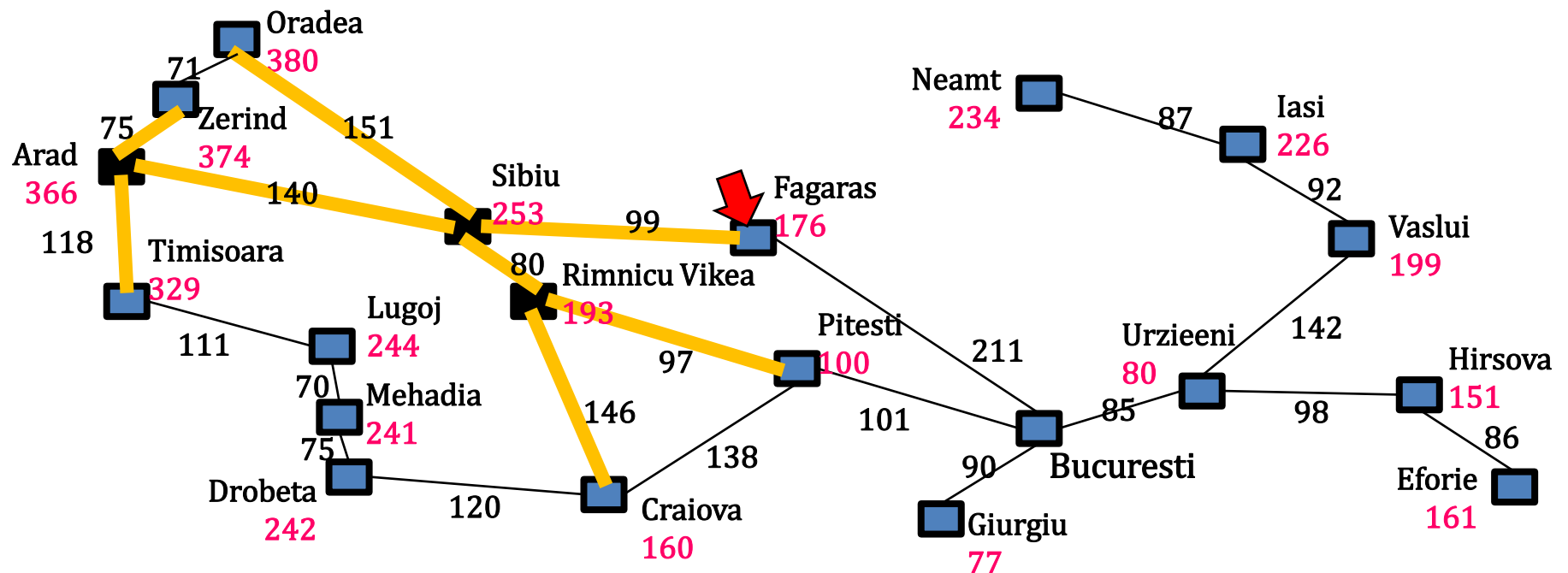
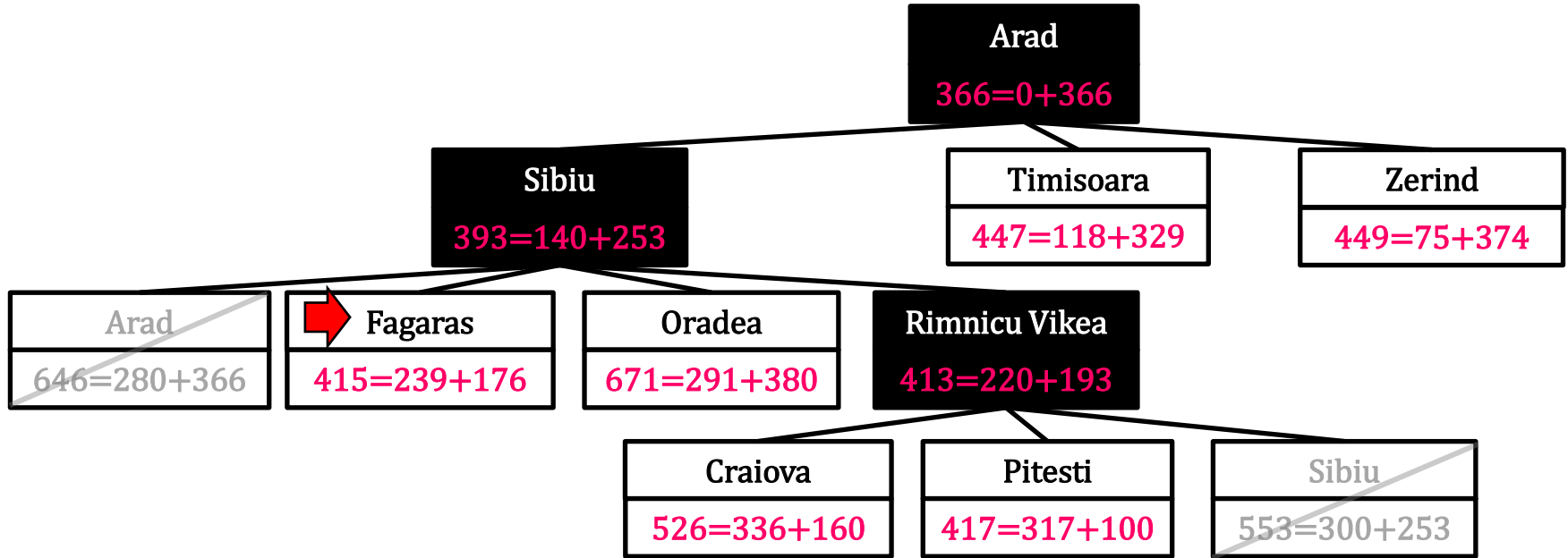
A*搜索算法

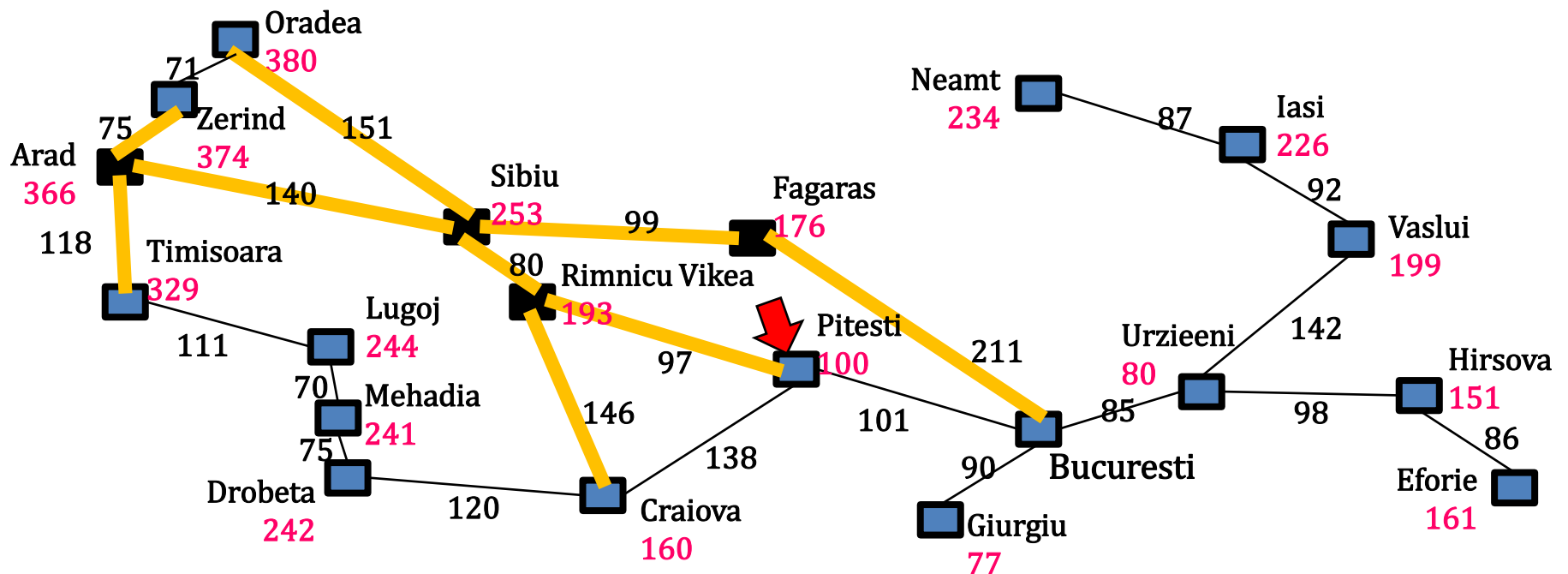
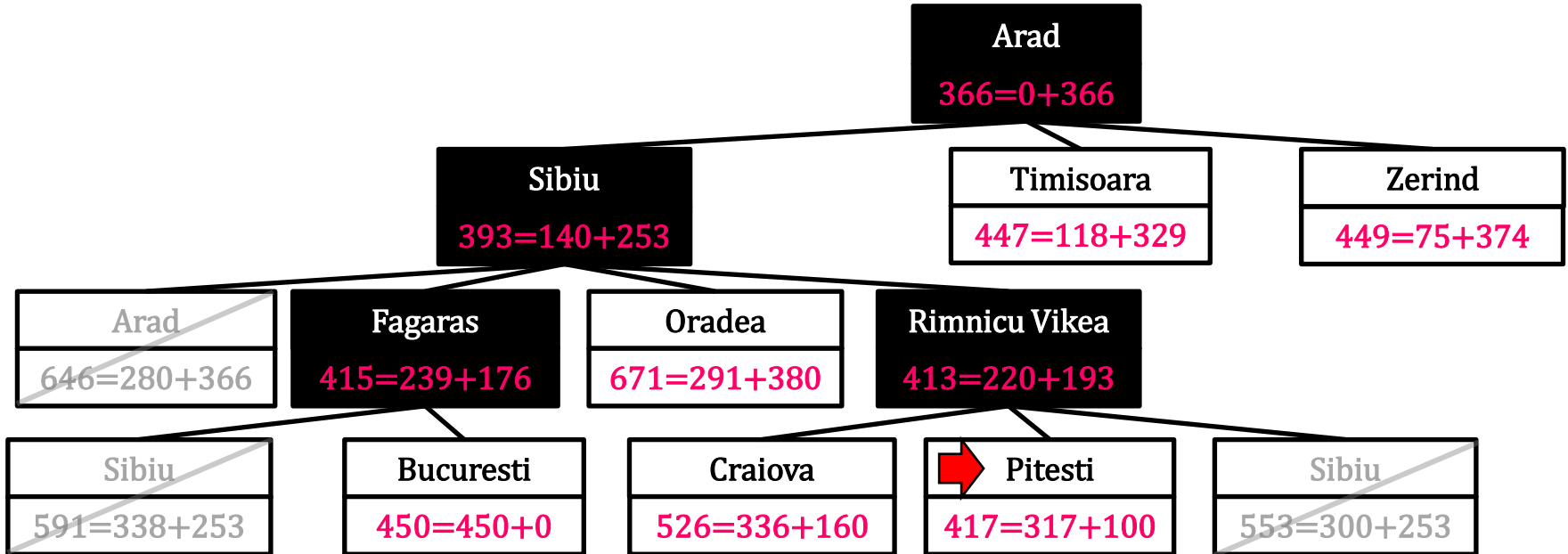
```
function A-Star-Search(problem) returns a solution, or failure
  node  $\leftarrow$  a node with State=problem.Initial-State, Path-Cost=0
  if problem.Goal-Test(node.State) then return Solution(node)
  frontier  $\leftarrow$  {node}
  explored  $\leftarrow$  an empty set
  loop do
    if Empty?(frontier) then return failure
    node  $\leftarrow$  POP the node with the lowest  $g(n)+h(n)$  in frontier
    if problem.Goal-Test(node.State) then return Solution(node)
    add node.State to explored
    for each action in problem.Actions(node.State) do
      child  $\leftarrow$  Child-Node(problem, node, action)
      if child.State is not in explored or frontier then
        frontier  $\leftarrow$  Insert(child, frontier)
      else if child.State is in frontier with higher  $g+h$  then
        replace that frontier node with child
      [else delete child]
```

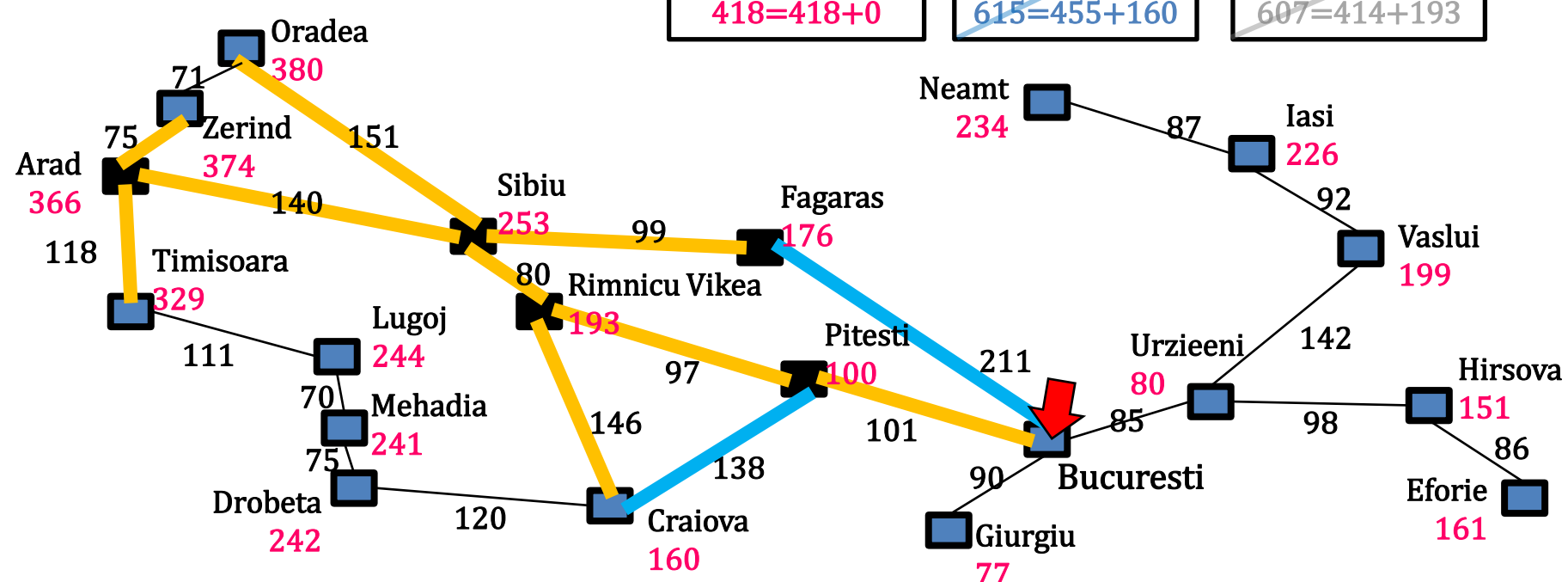
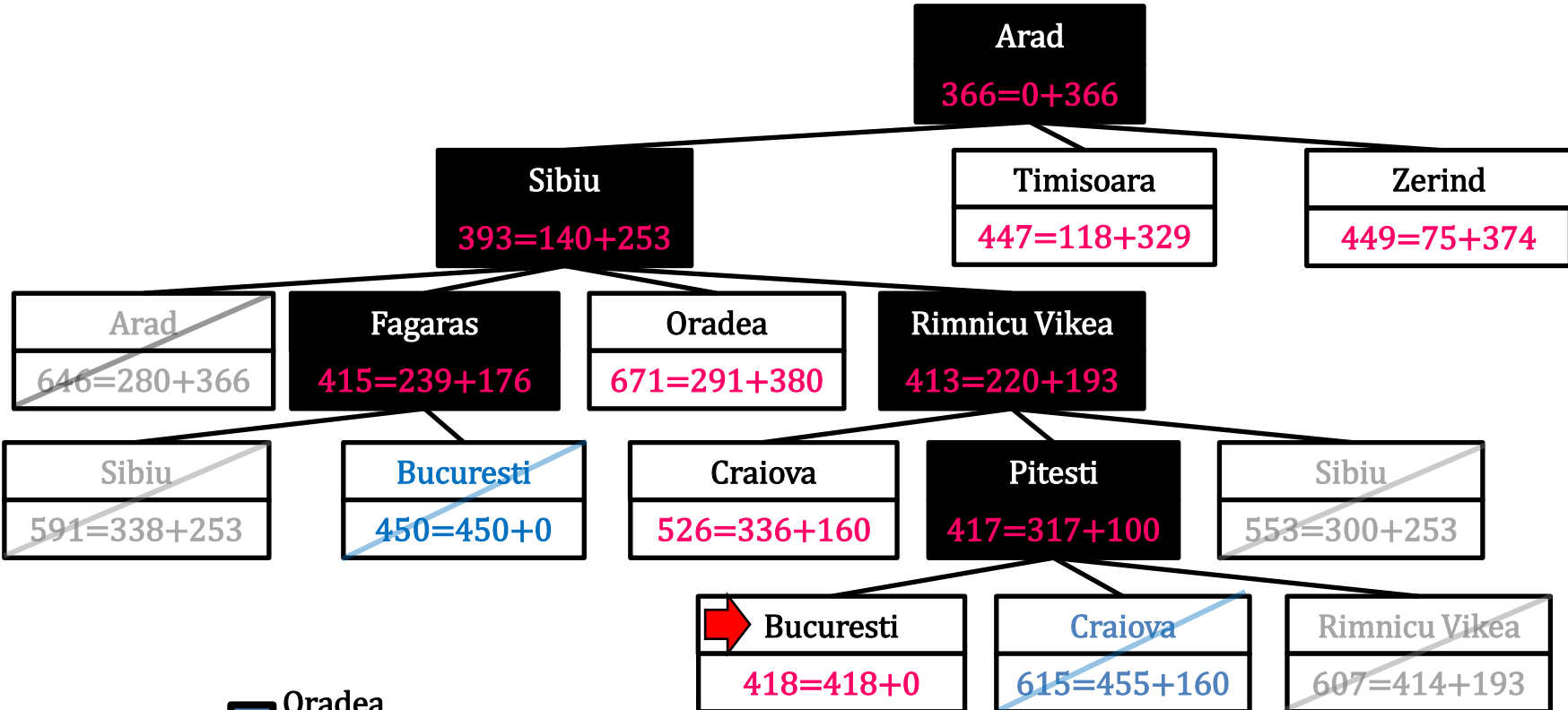
//Tree search:
Insert

similar to uniform cost search









Thanks

LU9UK2